

Importing the Libraries

```
# for manipulations
import numpy as np
import pandas as pd

# for data visualizations
import matplotlib.pyplot as plt
import seaborn as sns
plt.style.use('fivethirtyeight')

# for interactivity
import ipywidgets
from ipywidgets import interact
```

Reading the Dataset

```
# lets read the dataset
data = pd.read_csv('data.csv')

# lets check the shape of the dataset
print("Shape of the Dataset :", data.shape)
```

Shape of the Dataset : (2200, 8)

```
# lets check the head of the dataset
data.head()
```

	N	P	K	temperature	humidity	ph	rainfall	label
0	90	42	43	20.879744	82.002744	6.502985	202.935536	rice
1	85	58	41	21.770462	80.319644	7.038096	226.655537	rice
2	60	55	44	23.004459	82.320763	7.840207	263.964248	rice
3	74	35	40	26.491096	80.158363	6.980401	242.864034	rice
4	78	42	42	20.130175	81.604873	7.628473	262.717340	rice

```
# lets check if there is any missing value present in the dataset
data.isnull().sum()
```

N	0
P	0
K	0
temperature	0
humidity	0
ph	0
rainfall	0
label	0

dtype: int64

```
# lets check the Crops present in this Dataset
data['label'].value_counts()
```

pomegranate	100
lentil	100
coffee	100
mothbeans	100
grapes	100
orange	100
muskmelon	100
watermelon	100
blackgram	100
banana	100
cotton	100
kidneybeans	100
jute	100
pigeonpeas	100
papaya	100
coconut	100
mango	100
apple	100
rice	100
maize	100
chickpea	100
mungbean	100

Name: label, dtype: int64

Descriptive Statistics

```
# lets check the Summary for all the crops

print("Average Ratio of Nitrogen in the Soil :
{0:.2f}".format(data['N'].mean()))
print("Average Ratio of Phosphorous in the Soil :
{0:.2f}".format(data['P'].mean()))
print("Average Ratio of Potassium in the Soil :
{0:.2f}".format(data['K'].mean()))
print("Average Tempature in Celsius :
{0:.2f}".format(data['temperature'].mean()))
print("Average Relative Humidity in % :
{0:.2f}".format(data['humidity'].mean()))
print("Average PH Value of the soil :
{0:.2f}".format(data['ph'].mean()))
print("Average Rainfall in mm :
{0:.2f}".format(data['rainfall'].mean()))
```

Average Ratio of Nitrogen in the Soil : 50.55
Average Ratio of Phosphorous in the Soil : 53.36
Average Ratio of Potassium in the Soil : 48.15
Average Tempature in Celsius : 25.62
Average Relative Humidity in % : 71.48
Average PH Value of the soil : 6.47
Average Rainfall in mm : 103.46

```

# lets check the Summary Statistics for each of the Crops

@interact
def summary(crops = list(data['label'].value_counts().index)):
    x = data[data['label'] == crops]
    print("-----")
    print("Statistics for Nitrogen")
    print("Minimum Nitrogen required :", x['N'].min())
    print("Average Nitrogen required :", x['N'].mean())
    print("Maximum Nitrogen required :", x['N'].max())
    print("-----")
    print("Statistics for Phosphorous")
    print("Minimum Phosphorous required :", x['P'].min())
    print("Average Phosphorous required :", x['P'].mean())
    print("Maximum Phosphorous required :", x['P'].max())
    print("-----")
    print("Statistics for Potassium")
    print("Minimum Potassium required :", x['K'].min())
    print("Average Potassium required :", x['K'].mean())
    print("Maximum Potassium required :", x['K'].max())
    print("-----")
    print("Statistics for Temperature")
    print("Minimum Temperature required :
{0:.2f}".format(x['temperature'].min()))
    print("Average Temperature required :
{0:.2f}".format(x['temperature'].mean()))
    print("Maximum Temperature required :
{0:.2f}".format(x['temperature'].max()))
    print("-----")
    print("Statistics for Humidity")
    print("Minimum Humidity required :
{0:.2f}".format(x['humidity'].min()))
    print("Average Humidity required :
{0:.2f}".format(x['humidity'].mean()))
    print("Maximum Humidity required :
{0:.2f}".format(x['humidity'].max()))
    print("-----")
    print("Statistics for PH")
    print("Minimum PH required : {0:.2f}".format(x['ph'].min()))
    print("Average PH required : {0:.2f}".format(x['ph'].mean()))
    print("Maximum PH required : {0:.2f}".format(x['ph'].max()))
    print("-----")
    print("Statistics for Rainfall")
    print("Minimum Rainfall required :
{0:.2f}".format(x['rainfall'].min()))
    print("Average Rainfall required :
{0:.2f}".format(x['rainfall'].mean()))
    print("Maximum Rainfall required :
{0:.2f}".format(x['rainfall'].max()))

```

```
{"model_id": "22a562b9d7c843abbf5d86853fd10092", "version_major": 2, "version_minor": 0}
```

Lets compare the Average Requirement for each crops with average conditions

```
@interact
```

```
def compare(conditions =
['N', 'P', 'K', 'temperature', 'ph', 'humidity', 'rainfall']):
    print("Average Value for", conditions, "is
{0:.2f}".format(data[conditions].mean()))
    print("-----")
    print("Rice : {0:.2f}".format(data[(data['label'] == 'rice')]
[conditions].mean()))
    print("Black Grams : {0:.2f}".format(data[data['label'] ==
'blackgram'][conditions].mean()))
    print("Banana : {0:.2f}".format(data[(data['label'] == 'banana')]
[conditions].mean()))
    print("Jute : {0:.2f}".format(data[data['label'] == 'jute']
[conditions].mean()))
    print("Coconut : {0:.2f}".format(data[(data['label'] ==
'coconut')][conditions].mean()))
    print("Apple : {0:.2f}".format(data[data['label'] == 'apple']
[conditions].mean()))
    print("Papaya : {0:.2f}".format(data[(data['label'] == 'papaya')]
[conditions].mean()))
    print("Muskmelon : {0:.2f}".format(data[data['label'] ==
'muskmelon'][conditions].mean()))
    print("Grapes : {0:.2f}".format(data[(data['label'] == 'grapes')]
[conditions].mean()))
    print("Watermelon : {0:.2f}".format(data[data['label'] ==
'watermelon'][conditions].mean()))
    print("Kidney Beans: {0:.2f}".format(data[(data['label'] ==
'kidneybeans')][conditions].mean()))
    print("Mung Beans : {0:.2f}".format(data[data['label'] ==
'mungbean'][conditions].mean()))
    print("Oranges : {0:.2f}".format(data[(data['label'] == 'orange')]
[conditions].mean()))
    print("Chick Peas : {0:.2f}".format(data[data['label'] ==
'chickpea'][conditions].mean()))
    print("Lentils : {0:.2f}".format(data[(data['label'] == 'lentil')]
[conditions].mean()))
    print("Cotton : {0:.2f}".format(data[data['label'] == 'cotton']
[conditions].mean()))
    print("Maize : {0:.2f}".format(data[(data['label'] == 'maize')]
[conditions].mean()))
    print("Moth Beans : {0:.2f}".format(data[data['label'] ==
'mothbeans'][conditions].mean()))
    print("Pigeon Peas : {0:.2f}".format(data[(data['label'] ==
'pigeonpeas')][conditions].mean()))
```

```

    print("Mango : {0:.2f}".format(data[data['label'] == 'mango']
[conditions].mean()))
    print("Pomegranate : {0:.2f}".format(data[(data['label'] ==
'pomegranate')][conditions].mean()))
    print("Coffee : {0:.2f}".format(data[data['label'] == 'coffee']
[conditions].mean()))

{"model_id":"10ad6ccf7598439e98e15c2b06be683d","version_major":2,"vers
ion_minor":0}

# lets make this funtion more Intuitive

@interact
def compare(conditions =
['N','P','K','temperature','ph','humidity','rainfall']):
    print("Crops which require greater than average", conditions,'\n')
    print(data[data[conditions] > data[conditions].mean()]
['label'].unique())
    print("-----")
    print("Crops which require less than average", conditions,'\n')
    print(data[data[conditions] <= data[conditions].mean()]
['label'].unique())

{"model_id":"51bc5659482b4da7b2cd760db25e20ba","version_major":2,"vers
ion_minor":0}

```

Analyzing Agricultural Conditions

Lets check the distribution of Agricultural Conditions

```

plt.rcParams['figure.figsize'] = (15, 7)

plt.subplot(2, 4, 1)
sns.distplot(data['N'], color = 'lightgrey')
plt.xlabel('Ratio of Nitrogen', fontsize = 12)
plt.grid()

plt.subplot(2, 4, 2)
sns.distplot(data['P'], color = 'skyblue')
plt.xlabel('Ratio of Phosphorous', fontsize = 12)
plt.grid()

plt.subplot(2, 4, 3)
sns.distplot(data['K'], color = 'darkblue')
plt.xlabel('Ratio of Potassium', fontsize = 12)
plt.grid()

plt.subplot(2, 4, 4)
sns.distplot(data['temperature'], color = 'black')
plt.xlabel('Temperature', fontsize = 12)

```

```

plt.grid()

plt.subplot(2, 4, 5)
sns.distplot(data['rainfall'], color = 'grey')
plt.xlabel('Rainfall', fontsize = 12)
plt.grid()

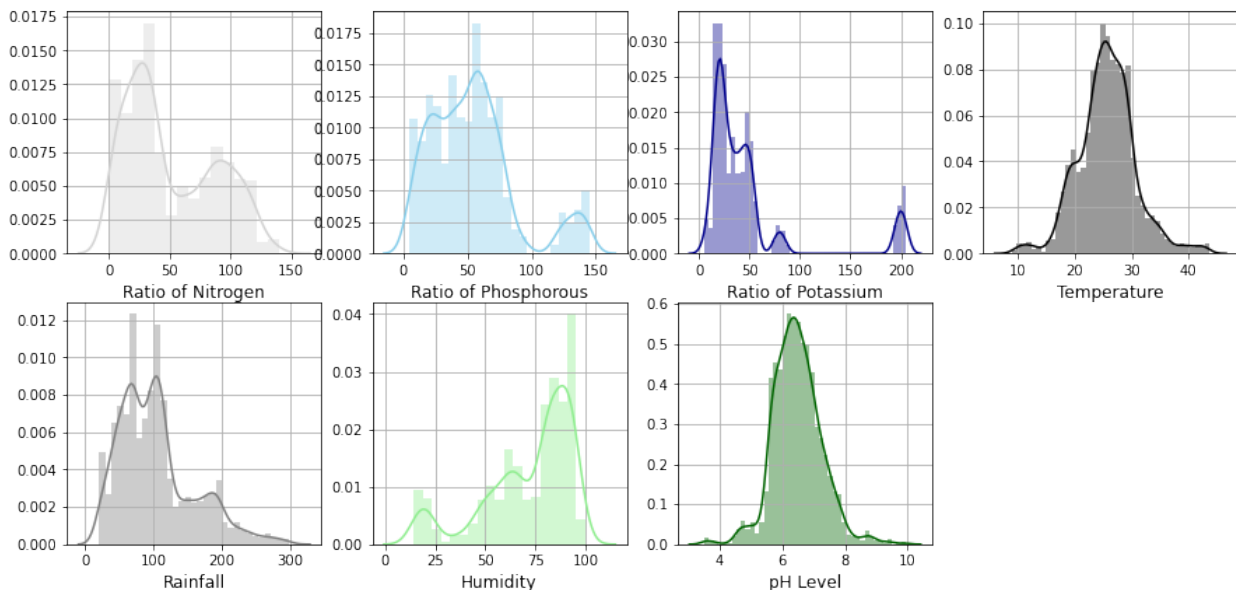
plt.subplot(2, 4, 6)
sns.distplot(data['humidity'], color = 'lightgreen')
plt.xlabel('Humidity', fontsize = 12)
plt.grid()

plt.subplot(2, 4, 7)
sns.distplot(data['ph'], color = 'darkgreen')
plt.xlabel('pH Level', fontsize = 12)
plt.grid()

plt.suptitle('Distribution for Agricultural Conditions', fontsize =
20)
plt.show()

```

Distribution for Agricultural Conditions



Lets find out some Interesting Facts

```

print("Some Interesting Patterns")
print("-----")
print("Crops which requires very High Ratio of Nitrogen Content in
Soil:", data[data['N'] > 120]['label'].unique())
print("Crops which requires very High Ratio of Phosphorous Content in
Soil:", data[data['P'] > 100]['label'].unique())

```

```

print("Crops which requires very High Ratio of Potassium Content in
Soil:", data[data['K'] > 200]['label'].unique())
print("Crops which requires very High Rainfall:",
data[data['rainfall'] > 200]['label'].unique())
print("Crops which requires very Low Temperature :",
data[data['temperature'] < 10]['label'].unique())
print("Crops which requires very High Temperature :",
data[data['temperature'] > 40]['label'].unique())
print("Crops which requires very Low Humidity:", data[data['humidity']
< 20]['label'].unique())
print("Crops which requires very Low pH:", data[data['ph'] < 4]
['label'].unique())
print("Crops which requires very High pH:", data[data['ph'] > 9]
['label'].unique())

```

Some Interesting Patterns

```

-----
Crops which requires very High Ratio of Nitrogen Content in Soil:
['cotton']
Crops which requires very High Ratio of Phosphorous Content in Soil:
['grapes' 'apple']
Crops which requires very High Ratio of Potassium Content in Soil:
['grapes' 'apple']
Crops which requires very High Rainfall: ['rice' 'papaya' 'coconut']
Crops which requires very Low Temperature : ['grapes']
Crops which requires very High Temperature : ['grapes' 'papaya']
Crops which requires very Low Humidity: ['chickpea' 'kidneybeans']
Crops which requires very Low pH: ['mothbeans']
Crops which requires very High pH: ['mothbeans']

```

Lets understand which crops can only be Grown in Summer Season, Winter Season and Rainy Season

```

print("Summer Crops")
print(data[(data['temperature'] > 30) & (data['humidity'] > 50)]
['label'].unique())
print("-----")
print("Winter Crops")
print(data[(data['temperature'] < 20) & (data['humidity'] > 30)]
['label'].unique())
print("-----")
print("Rainy Crops")
print(data[(data['rainfall'] > 200) & (data['humidity'] > 30)]
['label'].unique())

```

Summer Crops

```

['pigeonpeas' 'mothbeans' 'blackgram' 'mango' 'grapes' 'orange'
'papaya']

```

Winter Crops

```
['maize' 'pigeonpeas' 'lentil' 'pomegranate' 'grapes' 'orange']
-----
Rainy Crops
['rice' 'papaya' 'coconut']
```

Clustering Similar Crops

```
### Lets try to Cluster these Crops

# lets import the warnings library so that we can avoid warnings
import warnings
warnings.filterwarnings('ignore')

# Lets select the Spending score, and Annual Income Columns from the Data
x = data.loc[:,
['N', 'P', 'K', 'temperature', 'ph', 'humidity', 'rainfall']].values

# let's check the shape of x
print(x.shape)

# lets convert this data into a dataframe
x_data = pd.DataFrame(x)
x_data.head()

(2200, 7)
```

	0	1	2	3	4	5	6
0	90.0	42.0	43.0	20.879744	6.502985	82.002744	202.935536
1	85.0	58.0	41.0	21.770462	7.038096	80.319644	226.655537
2	60.0	55.0	44.0	23.004459	7.840207	82.320763	263.964248
3	74.0	35.0	40.0	26.491096	6.980401	80.158363	242.864034
4	78.0	42.0	42.0	20.130175	7.628473	81.604873	262.717340

```

# lets determine the Optimum Number of Clusters within the Dataset

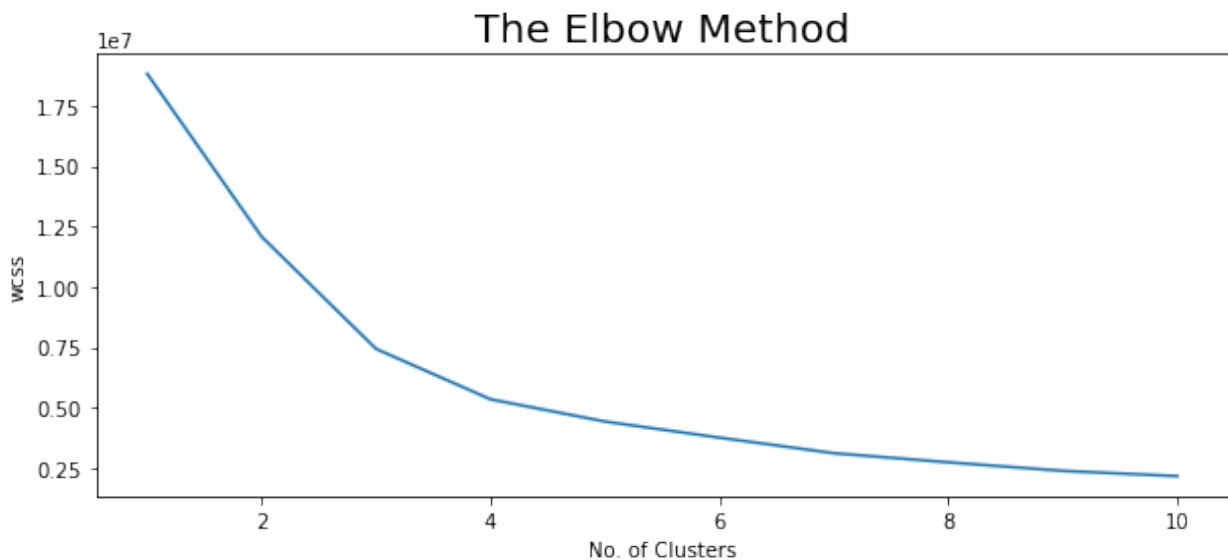
from sklearn.cluster import KMeans
plt.rcParams['figure.figsize'] = (10, 4)

wcss = []
for i in range(1, 11):
    km = KMeans(n_clusters = i, init = 'k-means++', max_iter = 300,
n_init = 10, random_state = 0)
    km.fit(x)
    wcss.append(km.inertia_)

# lets plot the results
plt.plot(range(1, 11), wcss)
plt.title('The Elbow Method', fontsize = 20)
plt.xlabel('No. of Clusters')
```



```
plt.ylabel('wcss')
plt.show()
```



```
# lets implement the K Means algorithm to perform Clustering analysis
km = KMeans(n_clusters = 4, init = 'k-means++', max_iter = 300, n_init
= 10, random_state = 0)
y_means = km.fit_predict(x)

# lets find out the Results
a = data['label']
y_means = pd.DataFrame(y_means)
z = pd.concat([y_means, a], axis = 1)
z = z.rename(columns = {0: 'cluster'})

# lets check the Clusters of each Crops
print("Lets check the Results After Applying the K Means Clustering
Analysis \n")
print("Crops in First Cluster:", z[z['cluster'] == 0]
['label'].unique())
print("-----")
print("Crops in Second Cluster:", z[z['cluster'] == 1]
['label'].unique())
print("-----")
print("Crops in Third Cluster:", z[z['cluster'] == 2]
['label'].unique())
print("-----")
print("Crops in Forth Cluster:", z[z['cluster'] == 3]
['label'].unique())
```

Lets check the Results After Applying the K Means Clustering Analysis

Crops in First Cluster: ['maize' 'chickpea' 'kidneybeans' 'pigeonpeas' 'mothbeans' 'mungbean' 'blackgram' 'lentil' 'pomegranate' 'mango' 'orange' 'papaya' 'coconut']

Crops in Second Cluster: ['grapes' 'apple']

Crops in Third Cluster: ['maize' 'banana' 'watermelon' 'muskmelon' 'papaya' 'cotton' 'coffee']

Crops in Forth Cluster: ['rice' 'pigeonpeas' 'papaya' 'coconut' 'jute' 'coffee']

Hard Clustering

```
print("Results for Hard Clustering\n")
counts = z[z['cluster'] == 0]['label'].value_counts()
d = z.loc[z['label'].isin(counts.index[counts >= 50])]
d = d['label'].value_counts()
print("Crops in Cluster 1:", list(d.index))
print("-----")
counts = z[z['cluster'] == 1]['label'].value_counts()
d = z.loc[z['label'].isin(counts.index[counts >= 50])]
d = d['label'].value_counts()
print("Crops in Cluster 2:", list(d.index))
print("-----")
counts = z[z['cluster'] == 2]['label'].value_counts()
d = z.loc[z['label'].isin(counts.index[counts >= 50])]
d = d['label'].value_counts()
print("Crops in Cluster 3:", list(d.index))
print("-----")
counts = z[z['cluster'] == 3]['label'].value_counts()
d = z.loc[z['label'].isin(counts.index[counts >= 50])]
d = d['label'].value_counts()
print("Crops in Cluster 4:", list(d.index))
```

Results for Hard Clustering

Crops in Cluster 1: ['pomegranate', 'kidneybeans', 'lentil', 'chickpea', 'blackgram', 'orange', 'mango', 'mothbeans', 'mungbean']

Crops in Cluster 2: ['grapes', 'apple']

Crops in Cluster 3: ['cotton', 'banana', 'maize', 'watermelon', 'muskmelon']

Crops in Cluster 4: ['jute', 'coconut', 'papaya', 'rice', 'coffee', 'pigeonpeas']

visualizing the Hidden Patterns

Data Visualizations

```
plt.rcParams['figure.figsize'] = (15, 8)

plt.subplot(2, 4, 1)
sns.barplot(data['N'], data['label'])
plt.ylabel(' ')
plt.xlabel('Ratio of Nitrogen', fontsize = 10)
plt.yticks(fontsize = 10)

plt.subplot(2, 4, 2)
sns.barplot(data['P'], data['label'])
plt.ylabel(' ')
plt.xlabel('Ratio of Phosphorous', fontsize = 10)
plt.yticks(fontsize = 10)

plt.subplot(2, 4, 3)
sns.barplot(data['K'], data['label'])
plt.ylabel(' ')
plt.xlabel('Ratio of Potassium', fontsize = 10)
plt.yticks(fontsize = 10)

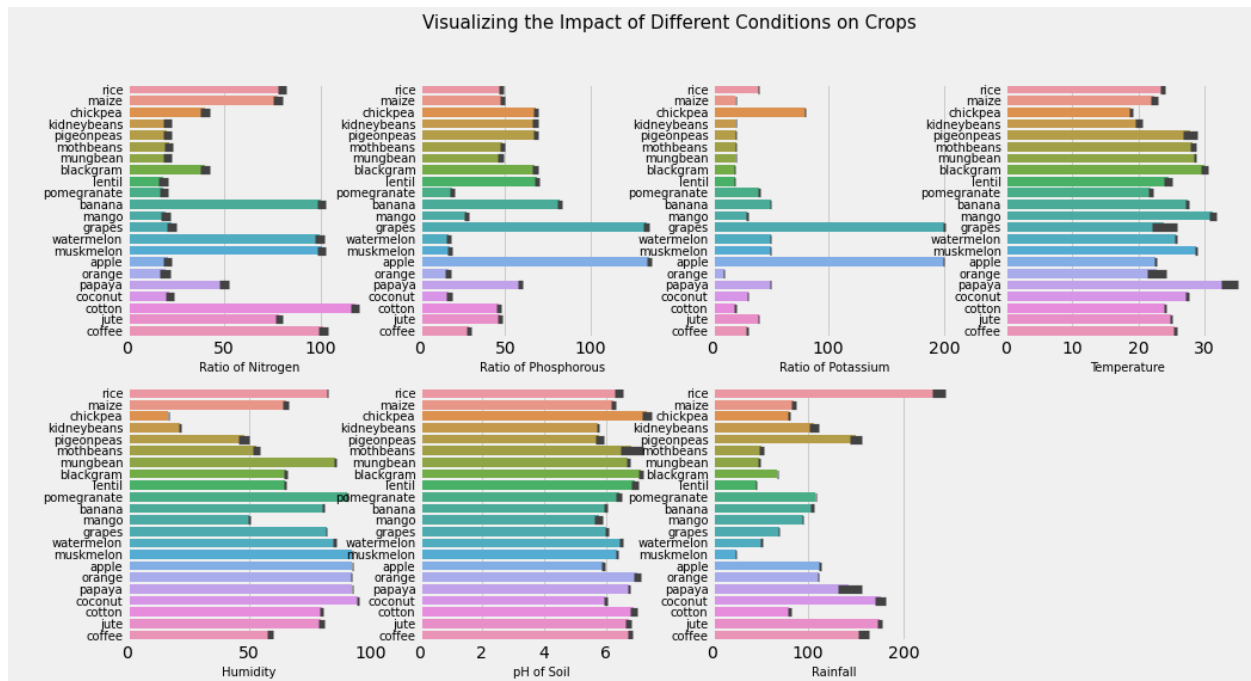
plt.subplot(2, 4, 4)
sns.barplot(data['temperature'], data['label'])
plt.ylabel(' ')
plt.xlabel('Temperature', fontsize = 10)
plt.yticks(fontsize = 10)

plt.subplot(2, 4, 5)
sns.barplot(data['humidity'], data['label'])
plt.ylabel(' ')
plt.xlabel('Humidity', fontsize = 10)
plt.yticks(fontsize = 10)

plt.subplot(2, 4, 6)
sns.barplot(data['ph'], data['label'])
plt.ylabel(' ')
plt.xlabel('pH of Soil', fontsize = 10)
plt.yticks(fontsize = 10)

plt.subplot(2, 4, 7)
sns.barplot(data['rainfall'], data['label'])
plt.ylabel(' ')
plt.xlabel('Rainfall', fontsize = 10)
plt.yticks(fontsize = 10)

plt.suptitle('Visualizing the Impact of Different Conditions on Crops', fontsize = 15)
plt.show()
```



Predictive Modelling

```
# lets split the Dataset for Predictive Modelling
```

```
y = data['label']
x = data.drop(['label'], axis = 1)
```

```
print("Shape of x:", x.shape)
print("Shape of y:", y.shape)
```

```
Shape of x: (2200, 7)
Shape of y: (2200,)
```

```
# lets create Training and Testing Sets for Validation of Results
from sklearn.model_selection import train_test_split
```

```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size =
0.2, random_state = 0)
```

```
print("The Shape of x train:", x_train.shape)
print("The Shape of x test:", x_test.shape)
print("The Shape of y train:", y_train.shape)
print("The Shape of y test:", y_test.shape)
```

```
The Shape of x train: (1760, 7)
The Shape of x test: (440, 7)
The Shape of y train: (1760,)
The Shape of y test: (440,)
```

```
# lets create a Predictive Model
```

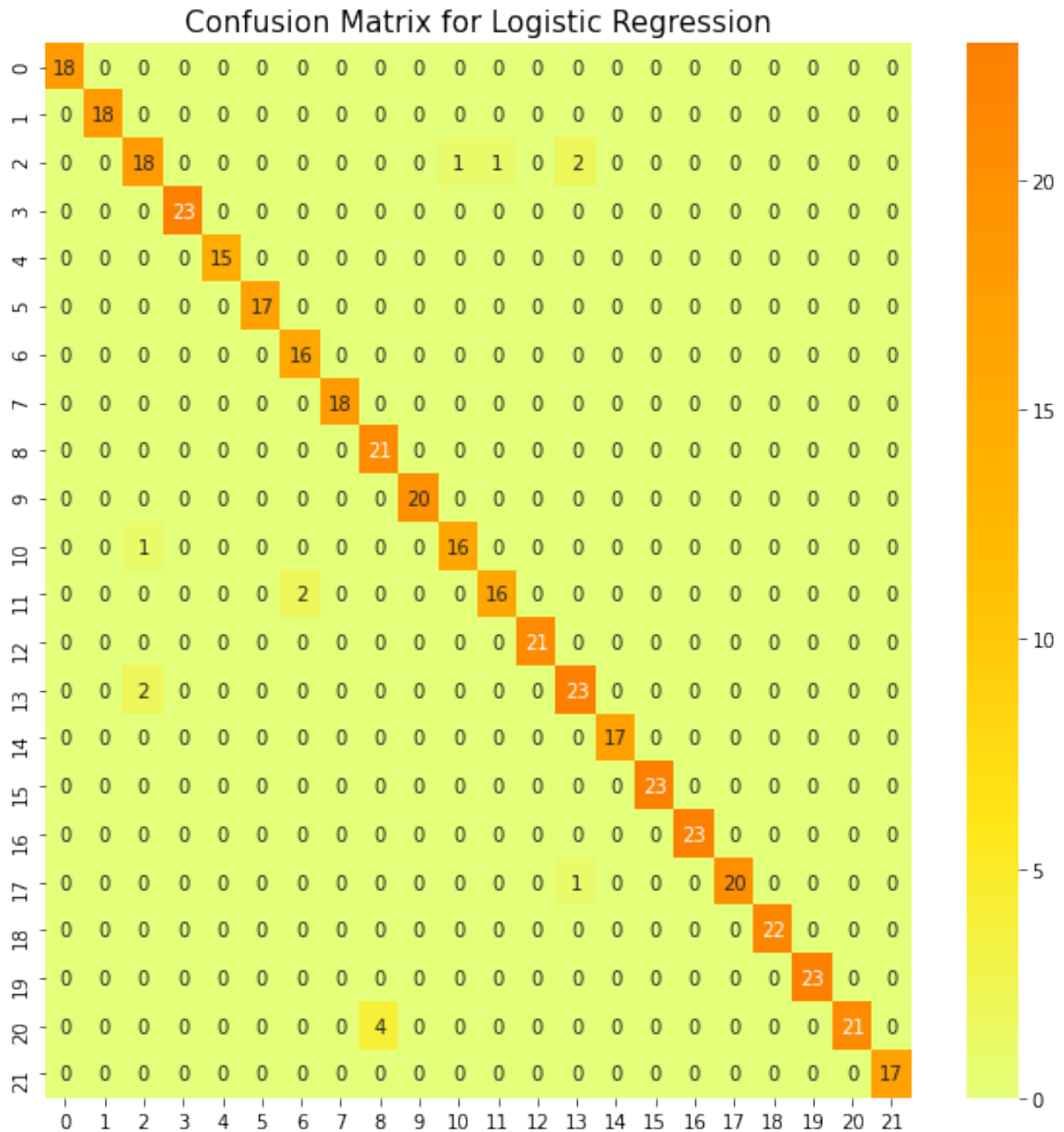
```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
model.fit(x_train, y_train)
y_pred = model.predict(x_test)

# lets evaluate the Model Performance
from sklearn.metrics import classification_report, confusion_matrix

# lets print the Confusion matrix first
plt.rcParams['figure.figsize'] = (10, 10)
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot = True, cmap = 'Wistia')
plt.title('Confusion Matrix for Logistic Regression', fontsize = 15)
plt.show()

# lets print the Classification Report also
cr = classification_report(y_test, y_pred)
print(cr)
```



	precision	recall	f1-score	support
apple	1.00	1.00	1.00	18
banana	1.00	1.00	1.00	18
blackgram	0.86	0.82	0.84	22
chickpea	1.00	1.00	1.00	23
coconut	1.00	1.00	1.00	15
coffee	1.00	1.00	1.00	17
cotton	0.89	1.00	0.94	16

grapes	1.00	1.00	1.00	18
jute	0.84	1.00	0.91	21
kidneybeans	1.00	1.00	1.00	20
lentil	0.94	0.94	0.94	17
maize	0.94	0.89	0.91	18
mango	1.00	1.00	1.00	21
mothbeans	0.88	0.92	0.90	25
mungbean	1.00	1.00	1.00	17
muskmelon	1.00	1.00	1.00	23
orange	1.00	1.00	1.00	23
papaya	1.00	0.95	0.98	21
pigeonpeas	1.00	1.00	1.00	22
pomegranate	1.00	1.00	1.00	23
rice	1.00	0.84	0.91	25
watermelon	1.00	1.00	1.00	17
accuracy				0.97
macro avg	0.97	0.97	0.97	440
weighted avg	0.97	0.97	0.97	440

Real time Predictions

lets check the Head of the Dataset

```
data.head()
```

	N	P	K	temperature	humidity	ph	rainfall	label
0	90	42	43	20.879744	82.002744	6.502985	202.935536	rice
1	85	58	41	21.770462	80.319644	7.038096	226.655537	rice
2	60	55	44	23.004459	82.320763	7.840207	263.964248	rice
3	74	35	40	26.491096	80.158363	6.980401	242.864034	rice
4	78	42	42	20.130175	81.604873	7.628473	262.717340	rice

```
prediction = model.predict((np.array([[90,
                                         40,
                                         40,
                                         20,
                                         80,
                                         7,
                                         200]])))
```

```
print("The Suggested Crop for Given Climatic Condition is :",
prediction)
```

The Suggested Crop for Given Climatic Condition is : ['rice']

lets check the Model for Oranges also

```
data[data['label'] == 'orange'].head()
```

	N	P	K	temperature	humidity	ph	rainfall	label
1600	22	30	12	15.781442	92.510777	6.354007	119.035002	orange
1601	37	6	13	26.030973	91.508193	7.511755	101.284774	orange

1602	27	13	6	13.360506	91.356082	7.335158	111.226688	orange
1603	7	16	9	18.879577	92.043045	7.813917	114.665951	orange
1604	20	7	9	29.477417	91.578029	7.129137	111.172750	orange

lets do some Real time Predictions

```
prediction = model.predict((np.array([[20,  
                                         30,  
                                         10,  
                                         15,  
                                         90,  
                                         7.5,  
                                         100]])))
```

```
print("The Suggested Crop for Given Climatic Condition is :",  
prediction)
```

The Suggested Crop for Given Climatic Condition is : ['orange']